

Python Interview Questions

Exception Handling & Context Managers — Top 100

try/except · finally · raise · Custom Exceptions · with · contextlib · Async · Patterns

Section 1 — Exception Fundamentals (Q1–Q25)

Q1. What is an exception in Python?

An exception is an event that disrupts the normal flow of a program. When Python encounters an error it cannot handle, it raises an exception object — an instance of a class derived from `BaseException`. If uncaught, the interpreter prints a traceback and terminates.

```
# Division by zero raises ZeroDivisionError
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f'Caught: {e}')    # Caught: division by zero

# Accessing missing key raises KeyError
d = {'a': 1}
try:
    val = d['b']
except KeyError as e:
    print(f'Key not found: {e}')    # Key not found: 'b'
```

Q2. What is the exception hierarchy in Python?

All exceptions inherit from `BaseException`. The main branches are: `SystemExit`, `KeyboardInterrupt`, `GeneratorExit` (direct `BaseException` subclasses — not caught by bare `'except Exception'`), and `Exception` (the base for all regular errors). Most user-defined exceptions should inherit from `Exception`.

```
# BaseException
# SystemExit
# KeyboardInterrupt
# GeneratorExit
# Exception
#   ArithmeticError -> ZeroDivisionError, OverflowError
#   LookupError     -> IndexError, KeyError
#   ValueError, TypeError, IOError, OSError ...

print(issubclass(ZeroDivisionError, ArithmeticError)) # True
print(issubclass(KeyError, LookupError))             # True
print(issubclass(Exception, BaseException))          # True
print(issubclass(SystemExit, Exception))             # False
```

Q3. What is the syntax of a try/except block?

try contains the code that may raise an exception. except catches specific exception types. else runs only when no exception was raised. finally always runs regardless of outcome.

```
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print('Cannot divide by zero')
        return None
    except TypeError as e:
        print(f'Type error: {e}')
        return None
    else:
        print('Division succeeded')
        return result
    finally:
        print('divide() finished')    # always runs

print(divide(10, 2))    # Division succeeded divide() finished 5.0
print(divide(10, 0))    # Cannot divide by zero divide() finished None
```

Q4. What is the difference between except Exception and bare except?

'except Exception' catches all regular exceptions but NOT SystemExit, KeyboardInterrupt, or GeneratorExit. Bare 'except:' catches everything including those — making it nearly impossible to interrupt the program with Ctrl+C. Always prefer 'except Exception'.

```
import sys

# BAD: swallows Ctrl+C and sys.exit()
# try:
#     ...code...
# except:
#     pass

# GOOD: catches all regular exceptions
try:
    raise RuntimeError('oops')
except Exception as e:
    print(f'Caught: {e}')    # Caught: oops

# SystemExit NOT caught by except Exception
try:
    raise SystemExit(1)
except Exception:
    print('This will NOT print')
except SystemExit:
    print('SystemExit caught separately')
```

Q5. What is the else clause in try/except?

The else block runs only if the try block completed without raising any exception. It separates 'success' logic from error-handling logic, making intent clearer than putting everything in try.

```
def read_int(s):
    try:
        value = int(s)
    except ValueError:
        print(f'Cannot convert {s!r} to int')
        return None
    else:
        # Only runs if int(s) succeeded
        print(f'Successfully converted: {value}')
        return value

read_int('42')    # Successfully converted: 42
read_int('abc')   # Cannot convert 'abc' to int
```

Q6. What is the finally clause and when does it run?

finally always executes — whether the try block succeeded, raised an exception, used return/break/continue, or even if the except block raised a new exception. It is the right place to release resources.

```
def risky(x):
    try:
        if x == 0: raise ValueError('zero')
        return 100 / x
    except ValueError as e:
        print(f'Error: {e}')
        return None
    finally:
        print('finally always runs')

print(risky(5))    # finally always runs    20.0
print(risky(0))    # Error: zero finally always runs    None

# Even with return inside try
def demo():
    try:
        return 'try'
    finally:
        print('finally ran!')    # still prints

demo()    # finally ran!
```

Q7. How do you raise an exception manually?

Use the raise statement with an exception class or instance. Raising a class auto-instantiates it. You can also re-raise the current exception with a bare raise inside an except block.

```
# Raise with message
def set_age(age):
    if not isinstance(age, int):
```

```

        raise TypeError(f'age must be int, got {type(age).__name__}')
    if age < 0 or age > 150:
        raise ValueError(f'age {age} is out of range [0,150]')
    return age

try:
    set_age(-5)
except ValueError as e:
    print(e)    # age -5 is out of range [0,150]

# Re-raise
try:
    1/0
except ZeroDivisionError:
    print('Logging error...')
    raise    # re-raises original exception

```

Q8. What is exception chaining (raise from)?

'raise NewError from original' sets the `__cause__` attribute, explicitly chaining exceptions. 'raise NewError' inside an except block implicitly chains via `__context__`. Use 'raise X from None' to suppress the chain.

```

# Explicit chaining
def load_config(path):
    try:
        with open(path) as f:
            return f.read()
    except OSError as e:
        raise RuntimeError(f'Failed to load {path}') from e

try:
    load_config('/no/such/file.cfg')
except RuntimeError as e:
    print(e)          # Failed to load /no/such/file.cfg
    print(e.__cause__) # [Errno 2] No such file...

# Suppress chain
try:
    int('x')
except ValueError:
    raise TypeError('bad input') from None    # __cause__ is None

```

Q9. How do you define a custom exception?

Subclass Exception (or a more specific exception class). Add `__init__` for custom attributes. Optionally override `__str__` for a custom message.

```

class AppError(Exception):
    '''Base class for application errors.'''

class ValidationError(AppError):
    def __init__(self, field, message):
        self.field = field
        self.message = message

```

```

        super().__init__(f'{field}: {message}')

class DatabaseError(AppError):
    def __init__(self, query, code):
        self.query = query
        self.code = code
        super().__init__(f'DB error {code} for query: {query}')

try:
    raise ValidationError('email', 'invalid format')
except ValidationError as e:
    print(e)          # email: invalid format
    print(e.field)    # email

```

Q10. What are the `__cause__`, `__context__`, and `__suppress_context__` attributes?

`__cause__`: set by 'raise X from Y' (explicit chain). `__context__`: set implicitly when exception raised during handling of another. `__suppress_context__`: True when 'raise X from None' used, hiding the implicit chain.

```

# Implicit context
try:
    try:
        1/0
    except ZeroDivisionError:
        raise ValueError('bad value') # __context__ = ZeroDivisionError
except ValueError as e:
    print(type(e.__context__)) # <class 'ZeroDivisionError'>
    print(e.__suppress_context__) # False

# Explicit cause
try:
    try:
        1/0
    except ZeroDivisionError as e:
        raise ValueError('bad') from e
except ValueError as e:
    print(type(e.__cause__)) # <class 'ZeroDivisionError'>

```

Q11. How do you catch multiple exception types in one except clause?

List multiple exception types in a tuple inside the except clause. This avoids code duplication when the handling logic is the same for several exceptions.

```

def parse(data):
    try:
        return int(data)
    except (ValueError, TypeError) as e:
        print(f'Parse error: {e}')
        return None

parse('abc') # Parse error: invalid literal...
parse(None) # Parse error: int() argument must be...
parse('42') # returns 42

```

```
# Multiple except clauses for different handling
try:
    val = {'key': 'value'}['missing']
except KeyError:
    print('Key missing')
except (TypeError, AttributeError):
    print('Type problem')
```

Q12. What is the difference between `except SomeError` and `except SomeError as e`?

'except SomeError' catches the exception but discards the object. 'except SomeError as e' binds the exception object to e, giving access to its message, attributes, and traceback. Note: 'e' is deleted from scope after the except block.

```
try:
    raise ValueError('detailed message', 42)
except ValueError as e:
    print(e.args)          # ('detailed message', 42)
    print(str(e))          # ('detailed message', 42)
    saved = e              # save before scope ends

# e is deleted here (CPython clears it after except block)
# print(e)                # NameError
print(saved)              # still accessible via saved
```

Q13. What is a traceback and how do you access it programmatically?

A traceback is the stack of frames at the point an exception was raised. Access it via `sys.exc_info()`, the `traceback` module, or the `__traceback__` attribute on the exception object.

```
import traceback, sys

def inner(): raise RuntimeError('boom')
def outer(): inner()

try:
    outer()
except RuntimeError as e:
    # Format as string
    tb_str = ''.join(traceback.format_exception(type(e), e, e.__traceback__))
    print(tb_str)

# sys.exc_info() returns (type, value, traceback)
exc_type, exc_val, exc_tb = sys.exc_info()
print(exc_type)    # <class 'RuntimeError'>
```

Q14. How do you log exceptions properly?

Use `logging.exception()` inside an except block — it logs the message AND the full traceback. Use `logging.error()` with `exc_info=True` for equivalent behaviour with a custom message.

```

import logging

logging.basicConfig(level=logging.DEBUG)
log = logging.getLogger(__name__)

def risky_operation():
    return 1 / 0

try:
    risky_operation()
except ZeroDivisionError:
    log.exception('Operation failed')    # logs traceback automatically

# Equivalent with exc_info
try:
    risky_operation()
except ZeroDivisionError:
    log.error('Custom message', exc_info=True)

```

Q15. What is warnings.warn() and how does it differ from raising an exception?

warnings.warn() issues a warning that by default is printed once per location but does NOT stop execution. Exceptions interrupt the flow. Warnings are used for deprecation notices or non-fatal issues.

```

import warnings

def old_api():
    warnings.warn('old_api is deprecated; use new_api()',
                  DeprecationWarning, stacklevel=2)
    return 'result'

result = old_api()    # DeprecationWarning printed, execution continues
print(result)        # result

# Turn warnings into errors for testing
warnings.filterwarnings('error', category=DeprecationWarning)
try:
    old_api()
except DeprecationWarning as e:
    print(f'Caught as error: {e}')

```

Q16. What is assert and when should you use it?

assert condition [, message] raises AssertionError if the condition is False. Use it for internal invariant checking during development. Never use assert for input validation in production — asserts are stripped with python -O.

```

def binary_search(arr, target):
    assert sorted(arr) == arr, 'Array must be sorted'
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1

```

```
        else: hi = mid - 1
    return -1

print(binary_search([1,3,5,7,9], 5))    # 2
# binary_search([3,1,2], 1)    # AssertionError: Array must be sorted
```

Q17. What is ExceptionGroup (Python 3.11+)?

ExceptionGroup wraps multiple unrelated exceptions that occurred simultaneously — common in async/concurrent code. Use 'except*' to handle specific types within the group.

```
# Python 3.11+
try:
    raise ExceptionGroup('multiple errors', [
        ValueError('bad value'),
        TypeError('bad type'),
        KeyError('missing key'),
    ])
except* ValueError as eg:
    print(f'ValueErrors: {eg.exceptions}')
except* (TypeError, KeyError) as eg:
    print(f'Type/Key errors: {eg.exceptions}')
```

Q18. How does exception handling affect performance?

Setting up a try block has negligible cost. The cost is paid when an exception is actually raised (creating the traceback is expensive). Use exceptions for exceptional conditions, not control flow in hot paths.

```
import timeit

# EAFP (Easier to Ask Forgiveness than Permission) – Pythonic
def get_eafp(d, key):
    try:
        return d[key]
    except KeyError:
        return None

# LBYL (Look Before You Leap)
def get_lbyl(d, key):
    if key in d:
        return d[key]
    return None

d = {'a': 1}
# Both similar speed when key exists; EAFP slower when exception fires
# For rare misses: EAFP. For frequent misses: LBYL.
```

Q19. What is the EAFP vs LBYL coding style?

EAFP (Easier to Ask Forgiveness than Permission) uses try/except and is the Pythonic style — try the operation, handle failure. LBYL (Look Before You Leap) checks preconditions with if statements. EAFP avoids race conditions in concurrent code.

```
data = {'name': 'Alice', 'age': 30}

# LBYL – defensive checking before access
if 'email' in data:
    email = data['email'].lower()
else:
    email = 'unknown'

# EAFP – attempt and handle failure
try:
    email = data['email'].lower()
except KeyError:
    email = 'unknown'

# EAFP with .get() shorthand (common Pythonic idiom)
email = (data.get('email') or '').lower() or 'unknown'
```

Q20. How do you suppress exceptions intentionally?

Use `contextlib.suppress(*exceptions)` to silently ignore specific exception types without a try/except block. This is cleaner than an empty except clause for known harmless errors.

```
from contextlib import suppress
import os

# Delete file if it exists (ignore if not found)
with suppress(FileNotFoundError):
    os.remove('/tmp/nonexistent_file.txt')

# Equivalent try/except (more verbose)
try:
    os.remove('/tmp/nonexistent_file.txt')
except FileNotFoundError:
    pass

# Suppress multiple
with suppress(KeyError, IndexError):
    val = {}['missing'][0]
print('After suppress – still running')
```

Q21. What is `sys.excepthook` and how do you use it?

`sys.excepthook` is called when an uncaught exception reaches the top level. You can replace it to log errors, display custom messages, or send crash reports — without wrapping every top-level call in try/except.

```
import sys, logging

logging.basicConfig(filename='crash.log', level=logging.CRITICAL)
```

```
def global_exception_handler(exc_type, exc_value, exc_tb):
    logging.critical('Uncaught exception',
                     exc_info=(exc_type, exc_value, exc_tb))
    # Call original handler to print traceback
    sys.__excepthook__(exc_type, exc_value, exc_tb)

sys.excepthook = global_exception_handler

# Now any unhandled exception will be logged to crash.log
```

Q22. How do you create an exception hierarchy for a library?

Define a base exception for your library and specific subclasses for different error categories. This lets users catch all library errors or specific ones.

```
class LibraryError(Exception):
    '''Base for all library errors.'''

class ConnectionError(LibraryError):
    def __init__(self, host, port, reason=''):
        self.host, self.port = host, port
        super().__init__(f'Cannot connect to {host}:{port}. {reason}')

class TimeoutError(LibraryError):
    def __init__(self, operation, timeout):
        self.operation = operation
        self.timeout = timeout
        super().__init__(f'{operation} timed out after {timeout}s')

# Catch all library errors
try:
    raise ConnectionError('db.local', 5432, 'refused')
except LibraryError as e:
    print(e)  # Cannot connect to db.local:5432. refused
```

Q23. What happens when an exception is raised in a finally block?

If finally raises an exception (or uses return/break), it replaces the original exception. The original exception is lost. This is a common, subtle bug — avoid raising exceptions in finally.

```
def dangerous_finally():
    try:
        raise ValueError('original')
    finally:
        raise RuntimeError('finally error')  # replaces ValueError!

try:
    dangerous_finally()
except RuntimeError as e:
    print(e)  # 'finally error'
    print(e.__context__)  # ValueError: original (set implicitly)

# Safe pattern: only release resources in finally
def safe():
```

```
resource = None
try:
    resource = acquire()
    do_work(resource)
finally:
    if resource: resource.release()    # no raise here
```

Q24. What is `traceback.format_exc()` and when do you use it?

`traceback.format_exc()` returns the current exception's traceback as a formatted string (what would be printed to `stderr`). Use it inside an `except` block to capture the traceback for logging or reporting.

```
import traceback

def process():
    raise RuntimeError('something broke')

try:
    process()
except Exception:
    tb = traceback.format_exc()
    print('Captured traceback:')
    print(tb)
    # Store in database, send to monitoring, etc.

    # Alternative: traceback.print_exc() prints directly to stderr
```

Q25. How do you test exception raising with `pytest`?

Use `pytest.raises()` as a context manager. Check the exception type, message, and attributes inside the `with` block.

```
import pytest

def divide(a, b):
    if b == 0:
        raise ZeroDivisionError('cannot divide by zero')
    return a / b

def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError, match='cannot divide'):
        divide(10, 0)

def test_exception_attributes():
    with pytest.raises(ZeroDivisionError) as exc_info:
        divide(5, 0)
    assert 'cannot divide' in str(exc_info.value)
    assert exc_info.type is ZeroDivisionError
```

Section 2 — Context Managers (Q26–Q55)

Q26. What is a context manager?

A context manager is an object that defines `__enter__` and `__exit__` methods. It is used with the `with` statement to guarantee setup and teardown logic runs around a block of code — even if an exception occurs.

```
class ManagedFile:
    def __init__(self, name, mode='r'):
        self.name = name
        self.mode = mode

    def __enter__(self):
        self.file = open(self.name, self.mode)
        return self.file          # value bound to 'as' variable

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        return False              # don't suppress exceptions

with ManagedFile('/tmp/test.txt', 'w') as f:
    f.write('hello')
# File automatically closed after with block
```

Q27. What are the `__enter__` and `__exit__` signatures?

`__enter__(self)` runs at the start of the `with` block and returns the context value (bound to the `as` variable). `__exit__(self, exc_type, exc_val, exc_tb)` runs at exit — all three are `None` if no exception occurred.

```
class Verbose:
    def __enter__(self):
        print('__enter__ called')
        return self          # 'as' variable gets self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f'__exit__ called: exc_type={exc_type}')
        return False         # False/None = do not suppress exception

with Verbose() as v:
    print(f'Inside with: {v}')
# __enter__ called
# Inside with: <Verbose ...>
# __exit__ called: exc_type=None
```

Q28. What does `__exit__` returning `True` mean?

If `__exit__` returns a truthy value, any exception raised inside the `with` block is suppressed (swallowed). Returning `False` or `None` re-raises it. Suppress only exceptions you understand and intend to handle.

```
class Suppressor:
    def __init__(self, *exc_types):
```

```

        self.exc_types = exc_types

    def __enter__(self): return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        # Return True to suppress matching exceptions
        return exc_type is not None and isinstance(exc_type, self.exc_types)

with Suppressor(ValueError, KeyError):
    raise ValueError('ignored')    # suppressed

print('Code continues after suppressed exception')

```

Q29. What is @contextlib.contextmanager?

@contextmanager converts a generator function with exactly one yield into a context manager. Code before yield is __enter__; code after yield is __exit__. Exceptions thrown in the with block appear at the yield point.

```

from contextlib import contextmanager
import time

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield                                # __enter__ returns here
    finally:
        elapsed = time.perf_counter() - start
        print(f'{label}: {elapsed:.4f}s')

with timer('sort 1M'):
    sorted(range(1_000_000))
# sort 1M: 0.0812s

```

Q30. How do you handle exceptions inside a @contextmanager?

Wrap the yield in try/except inside the generator. If you re-raise or don't catch, the exception propagates normally. If you catch and don't re-raise, it is suppressed.

```

from contextlib import contextmanager

@contextmanager
def safe_open(path):
    try:
        f = open(path)
        yield f
    except FileNotFoundError:
        print(f'File {path} not found - yielding None')
        yield None    # ERROR: can only yield once per contextmanager
    finally:
        # Cleanup always
        pass

```

```
# Correct pattern: single yield, handle around it
@contextmanager
def safe_section():
    try:
        yield
    except ZeroDivisionError:
        print('Division error suppressed')

with safe_section():
    x = 1 / 0    # suppressed
    print('continues')
```

Q31. What is `contextlib.suppress()`?

`contextlib.suppress(*exceptions)` is a context manager that suppresses the listed exception types. It is equivalent to `try/except: pass` but more readable for intentional suppression.

```
from contextlib import suppress
import os, json

# Silently ignore missing file
with suppress(FileNotFoundError):
    os.remove('nonexistent.txt')

# Silently ignore bad JSON
data = None
with suppress(json.JSONDecodeError):
    data = json.loads('bad json')
print(data)    # None – exception suppressed

# Multiple exception types
with suppress(KeyError, IndexError, AttributeError):
    val = [[]][0]    # IndexError suppressed
print('still running')
```

Q32. What is `contextlib.ExitStack`?

`ExitStack` dynamically stacks multiple context managers, entering them programmatically. Useful when the number of context managers is not known at compile time.

```
from contextlib import ExitStack

files = ['file1.txt', 'file2.txt', 'file3.txt']

# Open a variable number of files
with ExitStack() as stack:
    opened = [
        stack.enter_context(open(f, 'w'))
        for f in files
    ]
    for i, f in enumerate(opened):
        f.write(f'content {i}')
# All files closed automatically on exit
```

```
# Also useful for conditional context managers
with ExitStack() as stack:
    if True:
        lock = stack.enter_context(some_lock)
```

Q33. What is `contextlib.nullcontext()`?

`nullcontext(enter_result=None)` is a no-op context manager that optionally returns a value. Useful as a placeholder when a context manager is conditionally applied.

```
from contextlib import nullcontext
import threading

def process(data, thread_safe=False):
    ctx = threading.Lock() if thread_safe else nullcontext()
    with ctx:
        return [x*2 for x in data]

print(process([1,2,3], thread_safe=False)) # [2,4,6]
print(process([1,2,3], thread_safe=True))  # [2,4,6]

# With enter_result
with nullcontext('placeholder') as val:
    print(val) # placeholder
```

Q34. How do you nest context managers?

You can nest multiple `with` statements, or use a single `with` statement with comma-separated context managers (equivalent, but flatter). Both guarantee all `__exit__` methods are called.

```
# Nested with statements
with open('input.txt', 'w') as fin:
    with open('output.txt', 'w') as fout:
        fout.write(fin.read() if False else 'data')

# Flat: multiple context managers in one with (Python 2.7+)
with open('a.txt', 'w') as a, open('b.txt', 'w') as b:
    a.write('A'); b.write('B')

# Parenthesised (Python 3.10+) – cleaner for many managers
with (
    open('x.txt', 'w') as x,
    open('y.txt', 'w') as y,
):
    x.write('X'); y.write('Y')
```

Q35. How do you implement a database transaction context manager?

Commit on success, rollback on any exception in `__exit__`. The context manager encapsulates the transaction lifecycle.

```

class Transaction:
    def __init__(self, connection):
        self.conn = connection

    def __enter__(self):
        self.conn.begin()
        return self.conn

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is None:
            self.conn.commit()
        else:
            self.conn.rollback()
        return False # don't suppress exceptions

# Usage
# with Transaction(db_conn) as conn:
#     conn.execute('INSERT ...')
#     conn.execute('UPDATE ...')
# Commits if all succeed, rolls back on exception

```

Q36. How do you implement a timer/profiling context manager?

Record start time in `__enter__` and compute elapsed in `__exit__`. Using `@contextmanager` is more concise. Expose elapsed as an attribute for use after the with block.

```

import time
from contextlib import contextmanager
from dataclasses import dataclass, field

@dataclass
class Timing:
    label: str
    elapsed: float = field(default=0.0, init=False)

    def __enter__(self):
        self._start = time.perf_counter()
        return self

    def __exit__(self, *args):
        self.elapsed = time.perf_counter() - self._start
        print(f'{self.label}: {self.elapsed:.6f}s')
        return False

with Timing('sort') as t:
    sorted(range(100_000))
print(f'Elapsed: {t.elapsed:.6f}s') # accessible after with

```

Q37. How do you implement a temporary directory context manager?

`tempfile.TemporaryDirectory()` is the standard library solution. Implementing it manually demonstrates the pattern: create the directory in `__enter__`, remove it in `__exit__`.

```

import tempfile, shutil, os

```



```

from contextlib import contextmanager

@contextmanager
def temp_dir(suffix='', prefix='tmp'):
    path = tempfile.mkdtemp(suffix=suffix, prefix=prefix)
    try:
        yield path
    finally:
        shutil.rmtree(path, ignore_errors=True)

with temp_dir() as d:
    test_file = os.path.join(d, 'test.txt')
    with open(test_file, 'w') as f:
        f.write('temporary')
    print(os.listdir(d)) # ['test.txt']
# Directory and all contents deleted after with block

```

Q38. How do you implement a lock/mutex context manager?

`threading.Lock()` already implements the context manager protocol (`__enter__` acquires, `__exit__` releases). For custom locks, replicate this pattern.

```

import threading

lock = threading.Lock()
counter = 0

def increment():
    global counter
    with lock:
        # acquires on enter, releases on exit
        counter += 1

threads = [threading.Thread(target=increment) for _ in range(1000)]
[t.start() for t in threads]
[t.join() for t in threads]
print(counter) # 1000 - race-condition free

# Custom lock-like context manager
from contextlib import contextmanager
@contextmanager
def acquire(*locks):
    for lock in sorted(locks, key=id): lock.acquire()
    try: yield
    finally:
        for lock in reversed(locks): lock.release()

```

Q39. What is `contextlib.asynccontextmanager`?

`@asynccontextmanager` wraps an async generator function into an async context manager usable with `async with`. It mirrors `@contextmanager` but for async code.

```

from contextlib import asynccontextmanager
import asyncio

```

```

@asynccontextmanager
async def async_managed_conn(host):
    print(f'Connecting to {host}')
    conn = {'host': host, 'active': True}
    try:
        yield conn
    finally:
        conn['active'] = False
        print(f'Disconnected from {host}')

async def main():
    async with async_managed_conn('db.local') as conn:
        print(f'Using {conn}')

asyncio.run(main())
# Connecting to db.local Using {...} Disconnected from db.local

```

Q40. How do you implement an async context manager class?

Define `__aenter__` and `__aexit__` as async methods. They are the async equivalents of `__enter__` and `__exit__`.

```

import asyncio

class AsyncDB:
    def __init__(self, url): self.url = url; self.conn = None

    async def __aenter__(self):
        print(f'Connecting: {self.url}')
        await asyncio.sleep(0.01) # simulate IO
        self.conn = {'url': self.url}
        return self.conn

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        if exc_type:
            print(f'Rollback due to {exc_type.__name__}')
            print('Closing connection')
            self.conn = None
            return False

async def main():
    async with AsyncDB('postgresql://localhost/db') as conn:
        print('Querying...')

asyncio.run(main())

```

Q41. How do you reuse a context manager multiple times?

Most context managers are single-use (e.g., file objects are exhausted after closing). To reuse, re-enter the context or use `contextlib.contextmanager`'s generator which creates a fresh generator each call.

```

from contextlib import contextmanager

@contextmanager

```

```
def reusable_timer(label):
    import time
    start = time.perf_counter()
    yield
    print(f'{label}: {time.perf_counter()-start:.4f}s')

# Each call creates a new generator – fully reusable
with reusable_timer('first'):
    sum(range(100_000))

with reusable_timer('second'):
    sum(range(100_000))
```

Q42. How does the with statement work under the hood?

The with statement calls `__enter__`, executes the block, then calls `__exit__` with exception info (or `None, None, None`). If `__exit__` returns truthy, the exception is suppressed.

```
# What 'with ctx as val: body' compiles to:

mgr = ctx
val = mgr.__enter__()
exc = True
try:
    body # placeholder for with-block body
except:
    exc = False
    import sys
    if not mgr.__exit__(*sys.exc_info()):
        raise
finally:
    if exc:
        mgr.__exit__(None, None, None)
```

Q43. How do you implement a context manager that measures and limits memory?

Use `tracemalloc` in `__enter__` to start tracing and in `__exit__` to collect peak usage, raising an error if it exceeds a threshold.

```
import tracemalloc

class MemoryLimit:
    def __init__(self, max_mb):
        self.max_bytes = max_mb * 1024 * 1024

    def __enter__(self):
        tracemalloc.start()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        _, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        self.peak_mb = peak / (1024*1024)
        if exc_type is None and peak > self.max_bytes:
```

```

        raise MemoryError(f'Used {self.peak_mb:.1f}MB > {self.max_bytes/1e6}
        MB')
    return False

with MemoryLimit(max_mb=100) as m:
    data = list(range(100_000))
    print(f'Peak: {m.peak_mb:.2f}MB')

```

Q44. What is contextlib.closing()?

closing(obj) wraps any object with a close() method into a context manager that calls close() on exit. Useful for objects that have close() but don't implement __enter__/_exit__.

```

from contextlib import closing
import urllib.request

# urllib.response has close() but isn't always used as context manager
with closing(urllib.request.urlopen('http://example.com')) as resp:
    data = resp.read(100)
# resp.close() called automatically

# Custom class with close()
class Resource:
    def close(self): print('Resource closed')

with closing(Resource()) as r:
    print('Using resource')
# Resource closed

```

Q45. How do you implement a context manager for temporary environment variables?

Save, modify, and restore os.environ in __enter__ and __exit__. This is useful for tests that need specific environment settings.

```

import os
from contextlib import contextmanager

@contextmanager
def env_vars(**kwargs):
    old = {k: os.environ.get(k) for k in kwargs}
    os.environ.update({k: str(v) for k, v in kwargs.items()})
    try:
        yield
    finally:
        for k, v in old.items():
            if v is None:
                os.environ.pop(k, None)
            else:
                os.environ[k] = v

with env_vars(DEBUG='1', LOG_LEVEL='DEBUG'):
    print(os.environ['DEBUG']) # 1
print(os.environ.get('DEBUG')) # None - restored

```

Q46. How do you implement a context manager for mocking (patching)?

`unittest.mock.patch` is a context manager that replaces an attribute for the duration of the with block and restores the original on exit.

```
from unittest.mock import patch, MagicMock
import datetime

# Patch datetime.now() to return a fixed time
fixed = datetime.datetime(2025, 1, 1, 12, 0, 0)

with patch('datetime.datetime') as mock_dt:
    mock_dt.now.return_value = fixed
    result = datetime.datetime.now()
    print(result)    # 2025-01-01 12:00:00

# datetime.now works normally after with block
print(datetime.datetime.now() != fixed)    # True
```

Q47. How do you implement a context manager that redirects stdout?

`contextlib.redirect_stdout(new_target)` redirects `sys.stdout` inside the with block. Use `io.StringIO` to capture output as a string.

```
from contextlib import redirect_stdout
import io

# Capture printed output
buffer = io.StringIO()
with redirect_stdout(buffer):
    print('Hello, World!')
    print('Second line')

output = buffer.getvalue()
print(repr(output))    # 'Hello, World!\nSecond line\n'

# Also: contextlib.redirect_stderr for stderr
from contextlib import redirect_stderr
err_buf = io.StringIO()
with redirect_stderr(err_buf):
    import warnings
    warnings.warn('test', stacklevel=1)
```

Q48. How do you chain context managers with ExitStack for cleanup callbacks?

`ExitStack.callback()` registers an arbitrary callable to be called on exit — useful when you need to register cleanup without a full context manager object.

```
from contextlib import ExitStack
import tempfile, os

with ExitStack() as stack:
```

```
# Register a plain callback
stack.callback(print, 'Callback 1 ran')
stack.callback(print, 'Callback 2 ran')

# Enter a real context manager
f = stack.enter_context(open('/tmp/test.txt', 'w'))
f.write('data')

# On exit, callbacks run in LIFO order:
# Callback 2 ran
# Callback 1 ran
```

Q49. How do you implement a read-only attribute context manager?

Temporarily allow writes during `__enter__` by disabling write protection, perform operations, then restore protection in `__exit__`.

```
from contextlib import contextmanager

class Config:
    def __init__(self, **kw):
        object.__setattr__(self, '_data', dict(kw))
        object.__setattr__(self, '_locked', True)

    def __getattr__(self, k): return self._data[k]

    def __setattr__(self, k, v):
        if self._locked:
            raise AttributeError('Config is read-only')
        self._data[k] = v

    @contextmanager
    def editable(self):
        object.__setattr__(self, '_locked', False)
        try: yield self
        finally: object.__setattr__(self, '_locked', True)

cfg = Config(host='localhost', port=80)
with cfg.editable() as c:
    c.port = 443
print(cfg.port) # 443
# cfg.port = 80 # AttributeError
```

Q50. How do you write a context manager that retries on failure?

Use `@contextmanager` with a loop. The generator can yield inside a retry loop, and re-enter the loop if an exception occurs.

```
from contextlib import contextmanager
import time

@contextmanager
def retry_context(attempts=3, delay=0.1, exceptions=(Exception,)):
    last_error = None
```

```

for attempt in range(1, attempts + 1):
    try:
        yield attempt
        return # success – exit generator
    except exceptions as e:
        last_error = e
        print(f'Attempt {attempt} failed: {e}')
        if attempt < attempts:
            time.sleep(delay)
    raise last_error

import random
with retry_context(attempts=3, delay=0) as attempt:
    if random.random() < 0.7:
        raise IOError('transient error')
    print(f'Succeeded on attempt {attempt}')

```

Q51. How do you combine exception handling with context managers?

Context managers and try/except complement each other: use context managers for deterministic resource cleanup and try/except for error recovery logic. They can be freely combined.

```

from contextlib import contextmanager

@contextmanager
def managed_parse(raw):
    import json
    try:
        data = json.loads(raw)
        yield data
    except json.JSONDecodeError as e:
        print(f'Parse failed: {e}')
        yield None

with managed_parse('{"key": 1}') as data:
    if data: print(data['key']) # 1

with managed_parse('bad json') as data:
    print(data) # Parse failed: ... None

```

Q52. How does contextlib.AbstractContextManager work?

AbstractContextManager is an ABC that provides a default `__enter__` (returns self) and requires `__exit__` to be implemented. Useful as a base class for context managers.

```

from contextlib import AbstractContextManager

class LoggedContext(AbstractContextManager):
    def __init__(self, name): self.name = name

    # __enter__ is provided by ABC (returns self)

    def __exit__(self, exc_type, exc_val, exc_tb):
        status = 'OK' if exc_type is None else f'ERROR: {exc_type.__name__}'

```

```

        print(f'[{self.name}] {status}')
        return False

with LoggedContext('op1'):
    print('working')      # [op1] OK

with LoggedContext('op2'):
    raise ValueError('oops')  # [op2] ERROR: ValueError -> re-raised

```

Q53. What is contextlib.AbstractAsyncContextManager?

The async counterpart of AbstractContextManager. It provides a default `__aenter__` (returns self) and requires `__aexit__` to be implemented.

```

from contextlib import AbstractAsyncContextManager
import asyncio

class AsyncResource(AbstractAsyncContextManager):
    def __init__(self, name):
        self.name = name

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        await asyncio.sleep(0)
        print(f'Releasing {self.name}')
        return False

async def main():
    async with AsyncResource('conn') as r:
        print(f'Using {r.name}')  # Using conn
    # Releasing conn

asyncio.run(main())

```

Q54. How do you implement a context manager for a connection pool?

Acquire a connection from the pool in `__enter__` and return it in `__exit__`. Handle exceptions to ensure connections are always returned even on error.

```

from contextlib import contextmanager
from queue import Queue

class ConnectionPool:
    def __init__(self, size):
        self._pool = Queue(maxsize=size)
        for i in range(size):
            self._pool.put({'id': i, 'active': False})

    @contextmanager
    def acquire(self):
        conn = self._pool.get()  # blocks if pool empty
        conn['active'] = True
        try:
            yield conn
        finally:
            conn['active'] = False

```



```
        self._pool.put(conn) # always returned

pool = ConnectionPool(2)
with pool.acquire() as c:
    print(f'Using connection {c["id"]}') # 0
```

Q55. How do you write unit tests for context managers?

Test `__enter__` return value, that `__exit__` is called correctly, that resources are cleaned up, and that exceptions are properly propagated or suppressed.

```
from unittest.mock import MagicMock, patch, call
from contextlib import contextmanager
import pytest

class FileProcessor:
    def __enter__(self):
        self.file = open('/dev/null')
        return self
    def __exit__(self, *args):
        self.file.close()
        return False

def test_context_manager():
    with FileProcessor() as fp:
        assert fp.file is not None
    assert fp.file.closed # file closed after with

def test_exception_propagated():
    with pytest.raises(ValueError):
        with FileProcessor():
            raise ValueError('test')
```

Section 3 — Advanced Exception Patterns (Q56–Q75)

Q56. How do you implement a global exception handler for a Flask/web app?

Register error handlers using `@app.errorhandler`. For non-HTTP exceptions, use `@app.errorhandler(Exception)` as a catch-all.

```
# Flask error handling pattern
# from flask import Flask, jsonify
# app = Flask(__name__)

# @app.errorhandler(404)
# def not_found(e): return jsonify(error=str(e)), 404

# @app.errorhandler(ValidationError)
# def handle_validation(e):
#     return jsonify(error=e.message, field=e.field), 400
```

```
# @app.errorhandler(Exception)
# def handle_unexpected(e):
#     import logging
#     logging.exception('Unexpected error')
#     return jsonify(error='Internal server error'), 500

# Equivalent pattern using try/except in a base handler
class APIError(Exception):
    def __init__(self, message, code=400):
        self.message, self.code = message, code
        super().__init__(message)
```

Q57. How do you implement structured error responses in an API?

Define a hierarchy of API exceptions, each with HTTP status code and serialisation. Catch them in middleware and convert to JSON responses.

```
class APIError(Exception):
    status_code = 500
    def __init__(self, message, details=None):
        self.message = message
        self.details = details or {}
        super().__init__(message)
    def to_dict(self):
        return {'error': self.message, 'details': self.details,
                'status': self.status_code}

class NotFoundError(APIError):
    status_code = 404

class ValidationError(APIError):
    status_code = 422
    def __init__(self, field, message):
        super().__init__(message, {'field': field})

try:
    raise ValidationError('email', 'Invalid format')
except APIError as e:
    print(e.to_dict())
    # {'error': 'Invalid format', 'details': {'field': 'email'}, 'status': 422}
```

Q58. How do you implement exception handling in async code?

Use try/except inside async functions just like in synchronous code. For tasks, wrap with `asyncio.gather(return_exceptions=True)` to collect all results without short-circuiting on the first exception.

```
import asyncio

async def fetch(url):
    if 'bad' in url:
        raise ValueError(f'Bad URL: {url}')
    await asyncio.sleep(0.01)
```

```

        return f'data from {url}'

    async def main():
        urls = ['http://good.com', 'http://bad.com', 'http://also-good.com']

        results = await asyncio.gather(
            *[fetch(u) for u in urls],
            return_exceptions=True
        )

        for url, result in zip(urls, results):
            if isinstance(result, Exception):
                print(f'FAILED {url}: {result}')
            else:
                print(f'OK    {url}: {result[:20]}')

    asyncio.run(main())

```

Q59. What is the difference between OSError, IOError, and EnvironmentError?

In Python 3, IOError, EnvironmentError, WindowsError, and socket.error are all aliases for OSError. They are the same class. Use OSError (or more specific subclasses like FileNotFoundError, PermissionError) in new code.

```

print(IOError is OSError)          # True
print(EnvironmentError is OSError) # True

# Specific OSError subclasses (Python 3.3+)
# FileNotFoundError (errno ENOENT)
# PermissionError  (errno EACCES, EPERM)
# FileExistsError   (errno EEXIST)
# IsADirectoryError (errno EISDIR)
# TimeoutError      (errno ETIMEDOUT)

try:
    open('/no/such/file')
except FileNotFoundError as e:
    print(e.errno, e.strerror)    # 2 No such file or directory

```

Q60. How do you handle KeyboardInterrupt gracefully?

Catch KeyboardInterrupt at the top level to perform cleanup (save state, close connections) before exiting. Use finally or except KeyboardInterrupt specifically, and always re-raise or call sys.exit().

```

import sys, time

def long_running():
    print('Running... press Ctrl+C to stop')
    try:
        for i in range(10**9):
            time.sleep(0.1)
            print(f'Step {i}')
    except KeyboardInterrupt:
        print('\nInterrupted by user - cleaning up...')

```

```

        # Save state, close connections, etc.
    finally:
        print('Cleanup complete')

# long_running()    # run this to test

```

Q61. How do you implement a circuit breaker pattern?

Track consecutive failures. After a threshold, open the circuit (raise immediately without calling the function). Reset after a timeout.

```

import time
from contextlib import contextmanager

class CircuitBreaker:
    def __init__(self, max_failures=3, reset_timeout=30):
        self.max_failures = max_failures
        self.reset_timeout = reset_timeout
        self.failures = 0
        self.opened_at = None

    def call(self, fn, *args, **kwargs):
        if self.opened_at:
            if time.monotonic() - self.opened_at < self.reset_timeout:
                raise RuntimeError('Circuit open - fast-failing')
            self.opened_at = None; self.failures = 0
        try:
            result = fn(*args, **kwargs)
            self.failures = 0
            return result
        except Exception:
            self.failures += 1
            if self.failures >= self.max_failures:
                self.opened_at = time.monotonic()
                print(f'Circuit opened after {self.failures} failures')
            raise

cb = CircuitBreaker(max_failures=2, reset_timeout=5)
def bad(): raise IOError('down')
for _ in range(4):
    try: cb.call(bad)
    except Exception as e: print(e)

```

Q62. How do you implement exception-safe resource management without context managers?

Use try/finally to guarantee cleanup. This is the manual equivalent of `__enter__`/`__exit__` — verbose but necessary when context managers are not available.

```

# Manual try/finally pattern
resource = None
try:
    resource = open('/tmp/file.txt', 'w')
    resource.write('data')
    result = 'success'

```

```

except IOError as e:
    result = f'IO error: {e}'
    raise
finally:
    if resource is not None:
        resource.close() # always runs

# Better: use with statement
with open('/tmp/file.txt', 'w') as f:
    f.write('data')

```

Q63. What are pickling-related exceptions?

`pickle.PicklingError` and `pickle.UnpicklingError` are raised when objects cannot be serialised or deserialised. Lambdas, local functions, and objects with non-picklable state commonly cause these.

```

import pickle

# Lambda cannot be pickled
fn = lambda x: x + 1
try:
    pickle.dumps(fn)
except AttributeError as e:
    print(f'Cannot pickle: {e}')

# Custom class with non-picklable attribute
import threading
class Worker:
    def __init__(self):
        self.lock = threading.Lock() # not picklable

    def __getstate__(self): return {} # exclude lock
    def __setstate__(self, state):
        self.lock = threading.Lock() # recreate on load

w = Worker()
data = pickle.dumps(w) # works now
w2 = pickle.loads(data)
print(type(w2.lock)) # <class '_thread.lock'>

```

Q64. How do you implement a fallback chain for exceptions?

Try each strategy in order, catching exceptions and moving to the next. This is a common pattern for loading configuration or connecting to services with fallbacks.

```

def load_config_from_env():
    import os
    if 'APP_CONFIG' not in os.environ:
        raise EnvironmentError('APP_CONFIG not set')
    return {'source': 'env'}

def load_config_from_file():
    raise FileNotFoundError('config.json not found')

```

```
def load_config_default():
    return {'source': 'default', 'debug': False}

loaders = [load_config_from_env, load_config_from_file, load_config_default]

config = None
for loader in loaders:
    try:
        config = loader()
        print(f'Loaded from: {config["source"]}')
        break
    except (EnvironmentError, FileNotFoundError) as e:
        print(f'{loader.__name__} failed: {e}')

print(config)
```

Q65. How do you propagate exceptions across thread boundaries?

Threads swallow exceptions by default. Capture them in the thread and re-raise in the main thread via queue, concurrent.futures, or threading.excepthook.

```
import threading, queue

error_queue = queue.Queue()

def worker():
    try:
        raise RuntimeError('Thread crashed!')
    except Exception as e:
        error_queue.put(e)

t = threading.Thread(target=worker)
t.start()
t.join()

if not error_queue.empty():
    exc = error_queue.get()
    raise exc    # re-raise in main thread

# Cleaner: concurrent.futures handles this automatically
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor() as ex:
    fut = ex.submit(lambda: 1/0)
try:
    fut.result()    # ZeroDivisionError re-raised here
except ZeroDivisionError as e:
    print(f'Thread error: {e}')
```

Q66. How do you handle exceptions in generators?

Use try/except inside the generator body. The generator's throw() method injects exceptions at the yield point. Cleanup code goes in finally.

```
def safe_gen(data):
```

```

for item in data:
    try:
        yield int(item)
    except ValueError:
        print(f'Skipping invalid: {item!r}')

result = list(safe_gen(['1', 'two', '3', None, '5']))
# Skipping invalid: 'two'
# Skipping invalid: None
print(result)  # [1, 3, 5]

# Generator with finally for cleanup
def resource_gen():
    try:
        yield 'resource'
    finally:
        print('Generator cleanup')  # runs on close() or GC

```

Q67. What is the difference between raise and raise Exception()?

raise with no arguments inside an except block re-raises the current exception (preserving traceback). raise Exception() outside an except block creates a new bare exception with no information attached and a new traceback.

```

def example():
    try:
        1/0
    except ZeroDivisionError:
        print('Logging...')
        raise  # re-raises ZeroDivisionError with original traceback

try:
    example()
except ZeroDivisionError as e:
    print(f'Got: {e}')  # division by zero

# raise Exception() would start a new Exception, losing original context

```

Q68. How do you implement exception handling for a CLI application?

Catch exceptions at the top level, print user-friendly messages, and exit with appropriate codes. Use sys.exit() to return a non-zero code on error.

```

import sys

def main():
    try:
        # Application logic
        args = sys.argv[1:]
        if not args:
            raise ValueError('No arguments provided')
        result = int(args[0]) * 2
        print(result)
    except ValueError as e:
        print(f'Error: {e}', file=sys.stderr)

```

```

        sys.exit(1)
    except KeyboardInterrupt:
        print('\nAborted.', file=sys.stderr)
        sys.exit(130) # standard code for Ctrl+C
    except Exception as e:
        print(f'Unexpected error: {e}', file=sys.stderr)
        sys.exit(2)

if __name__ == '__main__':
    main()

```

Q69. How do you handle exceptions in context managers created with `__init_subclass__`?

Custom context manager base classes can define default `__exit__` behavior that subclasses inherit, allowing consistent exception handling across a family of context managers.

```

class BaseManager:
    def __enter__(self): return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.cleanup()
        if exc_type and issubclass(exc_type, OSError):
            print(f'OS error handled: {exc_val}')
            return True # suppress OSError
        return False

    def cleanup(self):
        print(f'{type(self).__name__} cleanup')

class FileManager(BaseManager):
    def __init__(self, path): self.path = path

with FileManager('/tmp/x') as fm:
    raise OSError('disk full') # suppressed by BaseManager
print('Continues after suppressed OSError')

```

Q70. What is `contextlib.aclosing()` and why is it important?

`aclosing(async_gen)` is an async context manager that calls `aclose()` on the async generator when the with block exits — ensuring cleanup code in finally blocks of async generators runs reliably.

```

import asyncio
from contextlib import aclosing

async def async_producer():
    try:
        for i in range(10):
            await asyncio.sleep(0)
            yield i
    finally:
        print('Producer cleaned up') # guaranteed by aclosing

async def main():

```



```

    async with aclosing(async_producer()) as gen:
        async for x in gen:
            if x == 3: break    # early exit – cleanup still runs
            print(x)

asyncio.run(main())
# 0 1 2 3  Producer cleaned up

```

Q71. How do you implement a saga/compensation pattern with exception handling?

A saga records completed steps. On failure, it runs compensating actions in reverse order to undo completed work.

```

class Saga:
    def __init__(self):
        self._steps = []

    def step(self, action, compensate):
        action()
        self._steps.append(compensate)

    def rollback(self):
        for comp in reversed(self._steps):
            try: comp()
            except Exception as e:
                print(f'Compensation failed: {e}')

log = []
saga = Saga()
try:
    saga.step(lambda: log.append('reserve_inventory'),
              lambda: log.append('release_inventory'))
    saga.step(lambda: log.append('charge_card'),
              lambda: log.append('refund_card'))
    raise RuntimeError('Shipment failed')    # trigger rollback
except RuntimeError:
    saga.rollback()
print(log)
# ['reserve_inventory', 'charge_card', 'refund_card', 'release_inventory']

```

Q72. How do you annotate exception types in Python type hints?

Use `raises` in docstrings (no runtime enforcement), or use third-party tools. Python has no native `'raises'` annotation. For return types, exceptions are not part of the type signature.

```

from typing import NoReturn

def fail_always(msg: str) -> NoReturn:
    '''Always raises RuntimeError.

    Raises:
        RuntimeError: always.
    '''
    raise RuntimeError(msg)

```

```
# NoReturn tells type checkers this function never returns normally
# Callers know the function either raises or loops forever

def parse_int(s: str) -> int:
    '''
    Raises:
        ValueError: if s cannot be converted to int.
    '''
    return int(s)
```

Q73. How do you implement a global unhandled promise/task exception handler in asyncio?

Set `loop.set_exception_handler()` to catch unhandled exceptions from asyncio tasks that were never awaited or whose exceptions were never retrieved.

```
import asyncio

def handle_task_exception(loop, context):
    msg = context.get('exception', context['message'])
    print(f'Unhandled task exception: {msg}')

async def bad_task():
    await asyncio.sleep(0.01)
    raise RuntimeError('Task failed!')

async def main():
    loop = asyncio.get_event_loop()
    loop.set_exception_handler(handle_task_exception)

    task = asyncio.create_task(bad_task())
    await asyncio.sleep(0.1) # let task run and fail
    # Exception handled by set_exception_handler

asyncio.run(main())
```

Q74. How do you implement a decorator that converts exceptions?

Catch specific exception types in the wrapper and re-raise them as different exception types, preserving the original via exception chaining.

```
from functools import wraps

def reraise_as(source_exc, target_exc):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            try:
                return fn(*args, **kwargs)
            except source_exc as e:
                raise target_exc(str(e)) from e
        return wrapper
    return decorator
```

```

class DBError(Exception): pass

@reraise_as(KeyError, DBError)
def get_record(d, key):
    return d[key]

try:
    get_record({'a': 1}, 'missing')
except DBError as e:
    print(f'DBError: {e}') # DBError: 'missing'
    print(type(e.__cause__)) # <class 'KeyError'>

```

Q75. What is the difference between contextmanager and a class-based context manager?

@contextmanager is concise for simple cases — the generator function captures state in local variables. A class-based CM is better when: state needs multiple attributes, methods need to be called during the block, or the CM needs to be serialisable/picklable.

```

# @contextmanager – concise, good for simple cases
from contextlib import contextmanager
@contextmanager
def simple_timer():
    import time
    t = time.perf_counter()
    yield
    print(f'Elapsed: {time.perf_counter()-t:.4f}s')

# Class-based – better when you need rich state/methods
import time
class RichTimer:
    def __enter__(self):
        self.start = time.perf_counter()
        self.laps = []
        return self
    def lap(self): # extra method only classes support
        self.laps.append(time.perf_counter() - self.start)
    def __exit__(self, *args):
        self.elapsed = time.perf_counter() - self.start

with RichTimer() as t:
    sum(range(10**5))
    t.lap()
    sum(range(10**5))
print(t.laps, t.elapsed)

```

Section 4 — Real-World Patterns & Tricky Questions (Q76–Q100)

Q76. How do you implement structured logging with exception context?

Attach structured metadata (request ID, user, operation) to exceptions so logs are searchable by context.

```
import logging, uuid
from contextlib import contextmanager

@contextmanager
def request_context(user=None):
    request_id = str(uuid.uuid4())[:8]
    log_extra = {'request_id': request_id, 'user': user or 'anon'}
    try:
        yield log_extra
    except Exception as e:
        logging.error(f'[{request_id}] Error: {e}', extra=log_extra,
                     exc_info=True)
        raise

with request_context(user='alice') as ctx:
    print(f'Processing request {ctx["request_id"]}')
    # raise ValueError('test') # would log with request_id
```

Q77. How do you implement a try/except inside a comprehension?

Comprehensions do not support try/except directly. Use a helper function with try/except, call it inside the comprehension.

```
def safe_convert(val, converter, default=None):
    try:
        return converter(val)
    except (ValueError, TypeError):
        return default

raw = ['1', 'two', '3.5', None, '42']

ints = [safe_convert(v, int) for v in raw]
floats = [safe_convert(v, float, 0.0) for v in raw]

print(ints) # [1, None, None, None, 42]
print(floats) # [1.0, 0.0, 3.5, 0.0, 42.0]
```

Q78. How do you implement exception handling for subprocess calls?

`subprocess.run()` raises `CalledProcessError` on non-zero exit if `check=True`. Handle it along with `FileNotFoundError` (bad command) and `TimeoutExpired`.

```
import subprocess
```

```
def run_command(cmd, timeout=30):
    try:
        result = subprocess.run(
            cmd, capture_output=True, text=True,
            timeout=timeout, check=True
        )
        return result.stdout
    except subprocess.CalledProcessError as e:
        print(f'Command failed (exit {e.returncode}): {e.stderr}')
        return None
    except subprocess.TimeoutExpired:
        print(f'Command timed out after {timeout}s')
        return None
    except FileNotFoundError:
        print(f'Command not found: {cmd[0]}')
        return None

output = run_command(['echo', 'hello'])
print(output)    # hello\n
```

Q79. How do you implement exception-safe cleanup with multiple resources?

Use `ExitStack` to open resources dynamically or a nested `try/finally` pattern. Exceptions during cleanup are collected and chained.

```
from contextlib import ExitStack

def process_files(paths):
    with ExitStack() as stack:
        files = [stack.enter_context(open(p)) for p in paths]
        # All files open; if any open() fails, already-opened are closed
        data = [f.read() for f in files]
    # All files closed here
    return data

# Create test files
import tempfile, os
paths = []
for content in ['hello', 'world']:
    f = tempfile.NamedTemporaryFile(mode='w', delete=False, suffix='.txt')
    f.write(content); f.close()
    paths.append(f.name)

print(process_files(paths))    # ['hello', 'world']
[os.unlink(p) for p in paths]
```

Q80. How do you implement defensive programming with exceptions?

Combine assertions (for internal invariants), explicit validation (for external input), typed exceptions (for categorisation), and logging at appropriate levels.

```
import logging
log = logging.getLogger(__name__)

class PaymentService:
```

```

def charge(self, amount: float, currency: str) -> str:
    # Validate external input
    if not isinstance(amount, (int, float)):
        raise TypeError(f'amount must be numeric, got {type(amount).__name__}')
    if amount <= 0:
        raise ValueError(f'amount must be positive, got {amount}')
    if currency not in ('USD', 'EUR', 'GBP'):
        raise ValueError(f'Unsupported currency: {currency}')

    # Internal invariant
    assert 0 < amount < 1_000_000, 'Extreme amount – check caller'

    try:
        return f'charged_{amount}_{currency}'
    except Exception as e:
        log.exception('Charge failed')
        raise

svc = PaymentService()
print(svc.charge(99.99, 'USD'))    # charged_99.99_USD

```

Q81. What is a try/except/else/finally execution order?

Execution order: try body -> (exception: except) -> (no exception: else) -> finally. The else only runs on success. The finally always runs. If except or else raises, finally still runs.

```

def demo(x):
    print('--- start ---')
    try:
        print('try')
        if x == 0: raise ValueError('zero')
        print('try continued')
    except ValueError as e:
        print(f'except: {e}')
    else:
        print('else (no exception)')    # only if no exception
    finally:
        print('finally (always)')      # always
    print('--- end ---')

demo(1)    # try  try continued  else (no exception)  finally (always)
print()
demo(0)    # try  except: zero   finally (always)

```

Q82. How do you implement timeouts for blocking operations?

Use `threading.Timer` to raise a `TimeoutError` after a deadline, or use `signal.alarm` (Unix only), or run the operation in a thread/process with a timeout join.

```

import threading, time

class TimeoutError(Exception): pass

def with_timeout(fn, args=(), timeout=5):

```

```

result = [None]
error = [None]

def target():
    try: result[0] = fn(*args)
    except Exception as e: error[0] = e

t = threading.Thread(target=target, daemon=True)
t.start()
t.join(timeout)

if t.is_alive():
    raise TimeoutError(f'Function timed out after {timeout}s')
if error[0]:
    raise error[0]
return result[0]

# Demo
result = with_timeout(sum, (range(10**6),), timeout=5)
print(result)    # 499999500000

```

Q83. How do you implement exception handling in a producer-consumer pattern?

Put exceptions into the queue alongside data. Consumers check whether they received an exception object and handle accordingly.

```

import queue, threading

SENTINEL = object()

def producer(q):
    for i in range(5):
        try:
            if i == 3: raise ValueError(f'Bad item {i}')
            q.put(('data', i))
        except Exception as e:
            q.put(('error', e))
    q.put(('done', SENTINEL))

def consumer(q):
    while True:
        kind, value = q.get()
        if kind == 'done': break
        elif kind == 'error':
            print(f'Received error: {value}')
        else:
            print(f'Processing: {value}')

q = queue.Queue()
threading.Thread(target=producer, args=(q,)).start()
consumer(q)

```

Q84. How do you handle exceptions in property setters?

Property setters can validate input and raise `ValueError` or `TypeError`. This is a common pattern for domain object invariants.

```
class Temperature:
    ABSOLUTE_ZERO = -273.15

    def __init__(self, celsius):
        self.celsius = celsius    # calls setter

    @property
    def celsius(self): return self._celsius

    @celsius.setter
    def celsius(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError(f'Expected number, got {type(value).__name__}')
        if value < self.ABSOLUTE_ZERO:
            raise ValueError(
                f'{value}C is below absolute zero ({self.ABSOLUTE_ZERO}C)')
        self._celsius = float(value)

t = Temperature(25)
print(t.celsius)    # 25.0
try:
    t.celsius = -300
except ValueError as e:
    print(e)    # -300C is below absolute zero...
```

Q85. How do you use `__enter__` and `__exit__` for mocking in tests?

Implement `__enter__` and `__exit__` on a mock object so it can be used as a context manager. `unittest.mock.MagicMock` supports this automatically.

```
from unittest.mock import MagicMock, patch

# MagicMock supports context manager protocol by default
mock_file = MagicMock()
mock_file.__enter__.return_value = mock_file
mock_file.__exit__.return_value = False

with patch('builtins.open', return_value=mock_file):
    with open('any.txt') as f:
        f.write('data')
    mock_file.write.assert_called_once_with('data')
    print('Mock assertions passed')
```

Q86. How do you implement exception handling for network requests?

Catch specific requests exceptions (`ConnectionError`, `Timeout`, `HTTPError`) and implement retry logic with exponential backoff.

```
import time

# Simulated requests-like exceptions
```



```

class RequestException(Exception): pass
class Timeout(RequestException): pass
class ConnectionError(RequestException): pass

def safe_request(url, retries=3, backoff=1.0):
    delay = backoff
    for attempt in range(1, retries + 1):
        try:
            # requests.get(url, timeout=10)
            if 'bad' in url: raise Timeout('Connection timed out')
            return {'status': 200, 'body': 'OK'}
        except Timeout as e:
            print(f'Attempt {attempt} timed out: {e}')
        except ConnectionError as e:
            print(f'Attempt {attempt} connection error: {e}')
        if attempt < retries:
            time.sleep(delay); delay *= 2
    raise RequestException(f'All {retries} attempts failed')

try:
    print(safe_request('http://good.com'))
except RequestException as e:
    print(f'Request failed: {e}')

```

Q87. What is the difference between `contextlib.contextmanager` and `contextlib.asynccontextmanager` yield semantics?

Both must yield exactly once. In `@contextmanager`, `yield` may appear inside `try/except/finally`. In `@asynccontextmanager`, the `yield` is inside `async def` — the function is an `async` generator and can use `await` before/after `yield`.

```

from contextlib import asynccontextmanager
import asyncio

@asynccontextmanager
async def rate_limited(calls_per_sec):
    interval = 1.0 / calls_per_sec
    last_call = [0.0]
    async def limiter():
        now = asyncio.get_event_loop().time()
        wait = interval - (now - last_call[0])
        if wait > 0:
            await asyncio.sleep(wait)
        last_call[0] = asyncio.get_event_loop().time()
    try:
        yield limiter
    finally:
        pass

async def main():
    async with rate_limited(2) as limit:
        await limit()
        print('Call 1')
        await limit()
        print('Call 2')

asyncio.run(main())

```

Q88. How do you implement exception handling for SQLAlchemy / ORM operations?

Wrap ORM calls in a session context manager. On exception, rollback; on success, commit. Map DB-specific exceptions to application-level exceptions.

```
from contextlib import contextmanager

# Simulated SQLAlchemy-like session
class Session:
    def begin(self):    print('BEGIN')
    def commit(self):   print('COMMIT')
    def rollback(self): print('ROLLBACK')
    def close(self):    print('CLOSE')
    def execute(self, q):
        if 'fail' in q: raise RuntimeError('DB error')
        return 'result'

@contextmanager
def session_scope(session):
    session.begin()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()

s = Session()
try:
    with session_scope(s) as sess:
        print(sess.execute('SELECT 1'))
        sess.execute('fail here')
except RuntimeError as e:
    print(f'Caught: {e}')
```

Q89. What is contextlib.redirect_stdout and how do you use it for testing?

`redirect_stdout(new_target)` redirects `sys.stdout`. Use with `io.StringIO()` to capture print output in tests without patching builtins.

```
from contextlib import redirect_stdout
import io

def print_report(data):
    print('=== Report ===')
    for k, v in data.items():
        print(f'{k}: {v}')

def test_print_report():
    buf = io.StringIO()
    with redirect_stdout(buf):
        print_report({'sales': 100, 'returns': 5})
    output = buf.getvalue()
    assert '=== Report ===' in output
    assert 'sales: 100' in output
```

```
assert 'returns: 5' in output
print('All assertions passed!')

test_print_report()
```

Q90. How do you handle exceptions when using `concurrent.futures`?

Exceptions raised in executor tasks are stored in the Future object and re-raised when `.result()` is called. Use `as_completed()` to process results as they finish, handling exceptions per-task.

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def task(n):
    if n == 3: raise ValueError(f'Task {n} failed')
    return n * n

with ThreadPoolExecutor(max_workers=4) as ex:
    futures = {ex.submit(task, i): i for i in range(6)}

    for fut in as_completed(futures):
        n = futures[fut]
        try:
            result = fut.result()
            print(f'task({n}) = {result}')
        except ValueError as e:
            print(f'task({n}) FAILED: {e}')
```

Q91. How do you implement a context manager for database connection retry?

Wrap connection acquisition in a retry loop inside `__enter__`. If all retries fail, raise the last exception.

```
import time
from contextlib import contextmanager

@contextmanager
def db_connection(host, retries=3, backoff=1.0):
    conn = None
    last_err = None
    delay = backoff
    for attempt in range(1, retries + 1):
        try:
            print(f'Connecting to {host} (attempt {attempt})')
            if host == 'bad': raise ConnectionRefusedError('refused')
            conn = {'host': host, 'open': True}
            break
        except ConnectionRefusedError as e:
            last_err = e
            if attempt < retries:
                time.sleep(delay); delay *= 2
    if conn is None:
        raise RuntimeError(f'Could not connect: {last_err}')
    try:
        yield conn
    finally:
        if conn: conn['open'] = False; print('Connection closed')
```

```
with db_connection('localhost') as c:
    print('Using:', c)
```

Q92. How do you implement a context manager for performance benchmarking?

Track wall time, CPU time, and memory delta. Expose as attributes after the with block for assertions in tests.

```
import time, tracemalloc
from dataclasses import dataclass, field

@dataclass
class Benchmark:
    label: str
    wall_time: float = field(default=0.0, init=False)
    cpu_time: float = field(default=0.0, init=False)
    peak_mb: float = field(default=0.0, init=False)

    def __enter__(self):
        tracemalloc.start()
        self._wall = time.perf_counter()
        self._cpu = time.process_time()
        return self

    def __exit__(self, *args):
        self.wall_time = time.perf_counter() - self._wall
        self.cpu_time = time.process_time() - self._cpu
        _, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        self.peak_mb = peak / (1024**2)
        print(f'{self.label}: wall={self.wall_time:.4f}s '
              f'cpu={self.cpu_time:.4f}s mem={self.peak_mb:.2f}MB')
        return False

with Benchmark('list build') as b:
    data = list(range(10**6))
    print(f'Assertions: wall<1s? {b.wall_time < 1}')

```

Q93. How do you use exceptions to implement flow control (anti-pattern)?

Using exceptions for normal control flow (e.g., to break out of loops) is an anti-pattern that hurts readability and performance. The Pythonic exception use is for exceptional, error conditions only.

```
# ANTI-PATTERN: using exception to exit loop
class Found(Exception):
    def __init__(self, value): self.value = value

def find_first(matrix, target):
    try:
        for i, row in enumerate(matrix):
            for j, val in enumerate(row):
                if val == target:
                    raise Found((i, j))    # bad practice
    except Found as e:

```

```

        return e.value
    return None

# BETTER: use a function and return early
def find_first_v2(matrix, target):
    for i, row in enumerate(matrix):
        for j, val in enumerate(row):
            if val == target:
                return (i, j)    # clean early return
    return None

m = [[1,2],[3,4],[5,6]]
print(find_first_v2(m, 4))    # (1, 1)

```

Q94. How do you implement a graceful shutdown with exception handling?

Register signal handlers (SIGINT, SIGTERM) that set a shutdown flag. The main loop checks the flag and performs cleanup before exiting.

```

import signal, time

class GracefulShutdown:
    def __init__(self):
        self.running = True
        signal.signal(signal.SIGINT, self._handle)
        signal.signal(signal.SIGTERM, self._handle)

    def _handle(self, signum, frame):
        print(f'\nSignal {signum} received – shutting down...')
        self.running = False

    def __enter__(self): return self
    def __exit__(self, *args):
        print('Cleanup complete')
        return False

with GracefulShutdown() as gs:
    print('Running – press Ctrl+C to stop')
    for _ in range(5):
        if not gs.running: break
        time.sleep(0.2)
    print('Exited cleanly')

```

Q95. How do you implement a context manager for temporary monkey-patching?

Save the original attribute in `__enter__` and restore it in `__exit__`. This is what `unittest.mock.patch` does internally.

```

from contextlib import contextmanager

@contextmanager
def patch_attr(obj, attr_name, new_value):
    original = getattr(obj, attr_name)
    setattr(obj, attr_name, new_value)

```

```

    try:
        yield
    finally:
        setattr(obj, attr_name, original)

import os
print(os.getcwd())    # real cwd

with patch_attr(os, 'getcwd', lambda: '/fake/path'):
    print(os.getcwd())    # /fake/path

print(os.getcwd())    # real cwd restored

```

Q96. What are common exception handling mistakes and how do you avoid them?

Common mistakes: bare except (swallows everything), catching Exception and not re-raising, catching too broadly, silently passing, and not logging. Always be specific, always log, always consider whether to re-raise.

```

# MISTAKE 1: bare except – catches SystemExit, KeyboardInterrupt
# try: ...
# except: pass    # NEVER do this

# MISTAKE 2: catching too broad without re-raising
# try: db.query()
# except Exception: pass    # silently swallows DB errors

# MISTAKE 3: not preserving exception chain
# except ValueError:
#     raise RuntimeError('failed')    # loses original cause

# CORRECT patterns:
import logging
log = logging.getLogger(__name__)

try:
    result = 1 / 0
except ZeroDivisionError as e:
    log.exception('Division failed')    # logs + traceback
    raise RuntimeError('compute failed') from e    # chain preserved

```

Q97. How do you implement a context manager that measures and reports errors?

Count successes and failures inside the context. Report metrics in `__exit__` — useful for batch processing loops.

```

from contextlib import contextmanager
from dataclasses import dataclass, field
from typing import List

@dataclass
class BatchStats:
    total: int = 0
    success: int = 0

```

```

errors: List[Exception] = field(default_factory=list)

@property
def failure_rate(self):
    return len(self.errors) / self.total if self.total else 0

@contextmanager
def batch_processor():
    stats = BatchStats()
    yield stats
    print(f'Processed: {stats.total}, OK: {stats.success}, '
          f'Errors: {len(stats.errors)}, '
          f'Failure rate: {stats.failure_rate:.1%}')

with batch_processor() as stats:
    for i in range(10):
        stats.total += 1
        try:
            if i % 3 == 0: raise ValueError(f'bad item {i}')
            stats.success += 1
        except ValueError as e:
            stats.errors.append(e)

```

Q98. How do you implement a decorator that adds retry with context manager support?

Combine a retry decorator with a context manager so retried code blocks can be expressed naturally.

```

from contextlib import contextmanager
from functools import wraps
import time

def retry(attempts=3, delay=0.1, exceptions=(Exception,)):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            last = None
            for i in range(1, attempts + 1):
                try:
                    return fn(*args, **kwargs)
                except exceptions as e:
                    last = e
                    print(f'Retry {i}: {e}')
                    if i < attempts: time.sleep(delay)
            raise last
        return wrapper
    return decorator

@retry(attempts=3, delay=0, exceptions=(ValueError,))
def parse(s):
    return int(s)

try:
    parse('bad')
except ValueError as e:
    print(f'All retries exhausted: {e}')

```

Q99. How do you implement exception handling for multiprocessing?

Exceptions in worker processes are pickled and re-raised in the parent when `.get()` is called on the result. Use `Pool.apply_async` with `error_callback` for async error handling.

```
from multiprocessing import Pool

def worker(n):
    if n == 3: raise ValueError(f'Worker failed for n={n}')
    return n * n

errors = []

def on_error(e):
    errors.append(e)
    print(f'Worker error: {e}')

with Pool(4) as pool:
    results = []
    for i in range(6):
        r = pool.apply_async(worker, args=(i,), error_callback=on_error)
        results.append(r)

    pool.close()
    pool.join()

    for i, r in enumerate(results):
        try:
            print(f'Result {i}: {r.get(timeout=1)}')
        except Exception as e:
            print(f'Result {i} failed: {e}')
```

Q100. Putting it all together — a production-grade exception handling and context manager system.

Combine custom exception hierarchy, context managers, retry logic, structured logging, graceful degradation, and metrics in a cohesive service layer.

```
import time, logging, functools
from contextlib import contextmanager
from dataclasses import dataclass, field
from typing import Optional, Type

logging.basicConfig(level=logging.INFO)
log = logging.getLogger('service')

# Exception hierarchy
class ServiceError(Exception): pass
class TransientError(ServiceError): pass
class PermanentError(ServiceError): pass

# Metrics
@dataclass
class Metrics:
    calls: int = 0
    errors: int = 0
    retries: int = 0
```



```

METRICS = Metrics()

# Retry decorator
def retry(attempts=3, delay=0.05, on=(TransientError,)):
    def dec(fn):
        @functools.wraps(fn)
        def wrapper(*a, **kw):
            METRICS.calls += 1
            for i in range(1, attempts + 1):
                try:
                    return fn(*a, **kw)
                except on as e:
                    METRICS.errors += 1
                    METRICS.retries += 1
                    log.warning(f'Transient error attempt {i}: {e}')
                    if i == attempts: raise PermanentError(str(e)) from e
                    time.sleep(delay)
            return wrapper
        return dec

# Context manager for operation tracing
@contextmanager
def operation(name):
    log.info(f'START {name}')
    t = time.perf_counter()
    try:
        yield
        log.info(f'OK {name} ({time.perf_counter()-t:.3f}s)')
    except ServiceError as e:
        log.error(f'FAIL {name}: {e}')
        raise
    except Exception as e:
        log.exception(f'UNEXPECTED {name}')
        raise ServiceError('unexpected') from e

# Service method
import random
@retry(attempts=3, delay=0)
def fetch_data(key):
    if random.random() < 0.4:
        raise TransientError('timeout')
    return f'data_{key}'

# Run
try:
    with operation('fetch_user_data'):
        result = fetch_data('user_123')
        print(f'Result: {result}')
except PermanentError as e:
    print(f'All retries failed: {e}')
except ServiceError as e:
    print(f'Service error: {e}')
finally:
    print(f'Metrics: {METRICS}')

```